

α -RACER: Real-Time Algorithm for Game-Theoretic Motion Planning and Control in Autonomous Racing using α -Potential Function

Dvij Kalaria*

EECS, UC Berkeley

DVIJK@BERKELEY.EDU

Chinmay Maheshwari*

EECS, UC Berkeley

CHINMAY_MAHESHWARI@BERKELEY.EDU

Shankar Sastry

EECS, UC Berkeley

SASTRY@COE.BERKELEY.EDU

Editors: N. Ozay, L. Balzano, D. Panagou, A. Abate

Abstract

Autonomous racing extends beyond the challenge of controlling a racecar at its physical limits. Similar to human drivers, autonomous vehicles must also employ strategic maneuvers, such as overtaking and blocking, to gain an advantage over competitors. While modern control algorithms can achieve human-level performance in single-car scenarios, research on real-time algorithms for multi-car autonomous racing remains limited. To bridge this gap, we develop a game-theoretic modeling framework that incorporates the competitive aspects of autonomous racing, such as overtaking and blocking, through a novel policy parametrization, while operating the car at its limit. We propose an algorithmic approach to compute an approximate Nash equilibrium strategy for our game model, representing the optimal strategy for any autonomous vehicle in a competitive environment. Our approach leverages a recently introduced framework of dynamic α -potential functions, enabling the real-time computation. Our approach comprises two phases: offline and online. During the offline phase, we use simulated racing data to learn an α -potential function that approximates utility changes for agents. This function facilitates the online computation of approximate Nash equilibria by maximizing its value. We evaluate our method in a head-to-head 3-car racing scenario, demonstrating superior performance over several existing baselines.

Website and Supplementary: <https://sastry-group.github.io/alpha-RACER/>

Keywords: Autonomous Multi-car Racing, Dynamic α -Potential Game, Nash Equilibrium.

1. Introduction

Autonomous racing is a challenging task in autonomous vehicle development, requiring efficient planning, reasoning, and action in high-speed, dynamic, and constrained environments—key for addressing edge cases in broader autonomous driving. Recent advances, such as deep reinforcement learning (RL), have enabled vehicles to outperform human drivers (Wurman et al., 2022; Kaufmann et al., 2023). However, challenges remain in optimizing strategies against other autonomous agents and in reducing the extensive training times required for achieving competitive performance.

A key challenge in multi-agent autonomous racing is developing real-time competitive strategies that consider the presence of other autonomous vehicles while maintaining high speeds (Betz et al., 2022). Algorithms must balance lap-time optimization with aggressive driving, collision avoidance, and dynamic responses to competitors. While existing research on single-agent racing focuses on

* Equal Contribution.

computing race lines based on vehicle dynamics and track constraints (Rosolia et al., 2017; Balaji et al., 2020; Song et al., 2021; Herman et al., 2021; Liniger et al., 2015; Kabzan et al., 2020), these approaches cannot accommodate complexities of multi-agent settings, where interactions such as blocking and overtaking become crucial. To address these challenges, multi-agent racing requires strategies that account for the interdependent behaviors of agents, with frameworks like Nash equilibrium offering a way to anticipate and adapt to competitors’ actions in a competitive environment.

Computation of Nash equilibrium is generally computationally challenging. Several studies have investigated its computation in autonomous racing, but many limitations persist. Many works use kinematic vehicle models (Notomista et al., 2020; Wang et al., 2019a, 2021; Liniger and Lygeros, 2019; Jia et al., 2023; Williams et al., 2017; Sinha et al., 2020; Schwarting et al., 2021), which simplify vehicle dynamics but fail to capture the nonlinear tire forces that are critical for high-speed racing. Others rely on open-loop control via receding horizon techniques (Buyval et al., 2017; Jung et al., 2021; Wang et al., 2019a, 2021; Liniger and Lygeros, 2019; Jia et al., 2023; Schwarting et al., 2021; Zhu and Borrelli, 2023, 2024), focusing on finite-horizon planning at the expense of long-term strategies. Additionally, many methods assume two-player zero-sum game (Wang et al., 2019a; Thakkar et al., 2022; Kalaria et al., 2023b; Zheng et al., 2022) or two-team competition (Thakkar et al., 2024), which are insufficient for races involving more than two agents. Alternative approaches, such as Stackelberg games (Notomista et al., 2020; Liniger and Lygeros, 2019), iterative linear quadratic games (Schwarting et al., 2021), and local Nash equilibrium (Zhu and Borrelli, 2023, 2024), provide partial local solutions but do not fully capture the global competitive nature of multi-agent racing.

Towards this end, we propose α -RACER (α -potential function based Real-time Algorithm for Competitive Racing). We study the following key question:

How to design real-time algorithms for autonomous racing that approximate Nash equilibrium, while accounting for nonlinear tire forces, long-horizon planning and accommodating competitive interaction in any number of competing vehicles?

We highlight three main contributions of this work towards answering the question posed above.

1. Modeling Contribution: We model the multi-agent interaction as an infinite-horizon discounted dynamic game and introduce a novel policy parametrization to enable competitive maneuvers. Specifically, we propose MPC-based policies that track a specially designed, parameterized *reference trajectory*, while avoiding other vehicles. This trajectory is derived by adjusting the optimal single-agent race-line to account for the presence of other agents, enabling competitive racing maneuvers such as overtaking and blocking. Additionally, we structure each agent’s immediate utility function to increase its relative progress along the track in comparison to other at each time-step. Moreover, our approach is modular with respect to the vehicle dynamics model. In this work, we use a nonlinear vehicle dynamics model that captures nonlinear tire forces, but the framework can also incorporate other advanced racing technologies, such as “push-to-pass”. By designing the reference trajectory in this way, we integrate long-term strategic planning for optimizing lap times with tactical planning for effective competition with other cars.

2. Algorithmic Contribution: We present an algorithmic approach to compute Nash equilibrium of the dynamic game by leveraging the recent framework of dynamic α -potential functions (developed in (Guo et al., 2023; Maheshwari et al., 2024; Guo et al., 2024)). Specifically, we compute an α -potential function that captures the change in each agent’s (long-term) value function resulting from unilateral deviations in its policy parameters. The structure of α -potential function allows

us to approximate the Nash equilibrium as an optimizer of this function (refer Proposition 3). We leverage this structure to develop a two-phase algorithmic approach to approximate the Nash equilibrium, consisting of an *offline* and an *online* phase. In the offline phase, we learn an α -potential function using simulated game data. During the online phase, the agent observes the current state of the game and selects the policy parameters that maximize the potential function in that state. This approach enables autonomous agents to engage in competitive maneuvers with reduced computational demands.

3. Numerical Evaluation: We numerically validate the effectiveness of our approach by applying it to three-car autonomous racing scenarios. Our results show that the approximation gap of our learned potential function is small, enabling us to closely approximate Nash equilibrium strategies. We demonstrate that the maximizer of the potential function effectively captures competitive racing maneuvers. Furthermore, we show that our method outperforms opponent policies—obtained using iterated best response, and finite-horizon self-play reinforcement learning—in most cases.

Other Related Works. Several works in the autonomous racing literature use the solution concept of (open-loop) *generalized Nash equilibrium* (e.g., (Wang et al., 2019a; Jia et al., 2023)) to incorporate hard collision avoidance constraints, which introduces added computational complexity. In contrast, our modeling framework incorporates collision avoidance in two ways: (i) through the MPC controller used to track the reference trajectory, and (ii) by slowing down vehicles when they enter a pre-defined radius around each other.

Our algorithmic approach also contributes to the growing literature on computational game theory for multi-robot systems (Laine et al., 2021; Cleac’h et al., 2019; Fridovich-Keil et al., 2020; Liu et al., 2023; Peters et al., 2024; Kavuncu et al., 2021; Sadigh et al., 2016; Wang et al., 2020a) by offering a new method to compute approximate Nash equilibrium.

Organization. In Section 2, we describe our model of multi-car racing as a dynamic game, including the policy parametrization, utility function, and vehicle dynamics model. In Section 3, we present our algorithmic approach. In Section 4, we evaluate the performance of our approach using a numerical racing simulator. Finally, we conclude with remarks in Section 5.

2. Modeling Multi-car Autonomous Racing

In this section, we first present the necessary preliminaries on dynamic game theory, followed by a novel model of multi-car autonomous racing as an instantiation of a dynamic game.

2.1. Preliminaries on Dynamic Game Theory

Consider a game involving N strategic players, with the set of players denoted as $\mathcal{N} := \{1, 2, \dots, N\}$. The game proceeds in discrete time steps, indexed by $t \in \mathbb{N}$. At each time step t , the state of player $i \in \mathcal{N}$ is represented as $\mathbf{x}_t^i \in \mathcal{X}^i$, where $\mathcal{X}^i$ is the set of all possible states for player i . The joint state of all players at time t is denoted by $\mathbf{x}_t = (\mathbf{x}_t^i)_{i \in \mathcal{N}} \in \mathcal{X}$, where $\mathcal{X} := \times_{i \in \mathcal{N}} \mathcal{X}^i$. At every time step t , each player $i \in \mathcal{N}$ selects an action $\mathbf{u}_t^i \in \mathcal{U}^i$, where $\mathcal{U}^i$ is the set of feasible actions for player i . The joint action at time t is expressed as $\mathbf{u}_t = (\mathbf{u}_t^i)_{i \in \mathcal{N}} \in \mathcal{U}$, where $\mathcal{U} := \times_{i \in \mathcal{N}} \mathcal{U}^i$.

For each player $i \in \mathcal{N}$, given the current state $\mathbf{x}_t^i$ and the joint action $\mathbf{u}_t$, the state transitions to a new state at time $t + 1$ according to the dynamics $\mathbf{x}_{t+1}^i = f^i(\mathbf{x}_t, \mathbf{u}_t^i)$, where $f^i : \mathcal{X} \times \mathcal{U}^i \rightarrow \mathcal{X}^i$ describes the state transition dynamics of player i . Finally, at each time step t , player $i \in \mathcal{N}$ receives

a reward $r_t^i(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1})$, which depends on the current and next joint state, and joint action of all players. We assume that players select their actions based on a state feedback strategy, denoted by $\pi^i : \mathcal{X} \rightarrow \mathcal{U}^i$, such that $\mathbf{u}_t^i = \pi^i(\mathbf{x}_t)$. The set of strategies of player $i \in \mathcal{N}$ is denoted by Π^i , and the set of joint strategies by $\Pi = \times_{i \in \mathcal{N}} \Pi^i$. In our study, we consider a parameterized set of policies. Specifically, for any player $i \in \mathcal{N}$, the player adopts a policy $\pi^i(\cdot; \theta^i) \in \Pi^i$, where $\theta^i \in \Theta^i$ represents the policy parameter. We consider non-myopic players who aim to maximize their long-term discounted utility starting from any state $\mathbf{x} \in \mathcal{X}$. Specifically, each player $i \in \mathcal{N}$ selects a parameter $\theta^i \in \Theta^i$ to optimize the discounted infinite-horizon objective, $V^i(\mathbf{x}, \theta^i, \theta^{-i}) := \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t^i(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1}) \right]$, where $\gamma \in [0, 1)$ is the discount factor, $\mathbf{x}_0 = \mathbf{x}$, $\mathbf{u}_t^i = \pi^i(\mathbf{x}_t; \theta^i)$, $\mathbf{u}_t^{-i} = \pi^{-i}(\mathbf{x}_t; \theta^{-i})^1$, and the state transitions according to $\mathbf{x}_{t+1} = f^i(\mathbf{x}_t, \mathbf{u}_t^i)$.

Definition 1 For every $\mathbf{x} \in \mathcal{X}$, a joint strategy profile $\theta^* \in \Theta$ is called a ϵ -Nash equilibrium if, for every $i \in \mathcal{N}$, $\theta^i \in \Theta^i$, $V^i(\mathbf{x}, \theta^{*,i}, \theta^{*, -i}) \geq V^i(\mathbf{x}, \theta^i, \theta^{*, -i}) - \epsilon$. If $\epsilon = 0$, the strategy profile θ^* is referred to simply as a Nash equilibrium.

2.2. Multi-car Racing as a Dynamic Game

In this subsection, we formulate the multi-car racing problem as a dynamic game by detailing the various components of the game.

Set of Players. Each car is modeled as a strategic player in the dynamic game.

Set of States and Actions. For every car $i \in \mathcal{N}$, the state $\mathbf{x}^i = (p_x^i, p_y^i, \phi^i, v_x^i, v_y^i, \omega^i)$, where (p_x^i, p_y^i) denote the longitudinal and lateral position of car i in the *Frenet frame* along the track; ϕ^i denotes the orientation of the car in the Frenet frame along the track; (v_x^i, v_y^i) denote the longitudinal and lateral velocities of car i in the Frenet frame; and ω^i denotes the angular velocity of car i in the body frame. Additionally, $\mathbf{u}^i = (d^i, \delta^i)$, where $\delta^i \in [\delta_{\min}, \delta_{\max}]$ denotes the steering angle of car i and $d^i \in [d_{\min}, d_{\max}]$ is the throttle input of car i .

Dynamics. The most widely used dynamics models in the context of racing are the *kinematic* (Jia et al., 2023; Notomista et al., 2020; Wang et al., 2019b, 2021) and *dynamic bicycle models* (Kalaria et al., 2023a; Brunke, 2020). In this work, we use the dynamic model as it can accurately model the high-speed maneuvers of the car. A detailed vehicle model description is in Supplementary. On top of the standard dynamic bicycle model, we also incorporate near-collision behavior in our dynamics. Suppose two cars, i and j , are within an unsafe distance from each other and $p_{x,t}^i > p_{x,t}^j$, we reduce their velocities to replicate the time lost due to collision in an actual race with more penalty for the car behind than the one ahead. More concretely, we update the dynamics as $v_{x,t+1}^i = (1/2) \cdot v_{x,t}^i$ and $v_{x,t+1}^j = (1/3) \cdot v_{x,t}^j$. Moreover, if a car goes out of track boundary, i.e., when $|p_{y,t}^i| > w_{\max}/2$ (where $w_{\max}$ is the width of track) we penalize it by reducing its speed and re-align the car along the track. More concretely, we update the dynamics as $v_{x,t+1}^i = v_{x,t}^i/2$ and $\phi_{t+1}^i = 0$.

Policy Parametrization. In this section, we introduce a *novel* policy parametrization $(\Theta^i)_{i \in \mathcal{N}}$ designed to capture competitive driving behaviors, such as overtaking, blocking, apex hugging, late braking, and early acceleration. To achieve this, we restrict the set of policies to MPC controllers that track a reference trajectory specifically designed to encode such competitive behaviors in multi-car racing. For each car $i \in \mathcal{N}$, the parametrization of the MPC controller and its reference tra-

1. We use the standard game theory notation of π^{-i} (resp. θ^{-i}) to denote the joint policy (joint policy parameters) of all players except player i .

jectory is represented by θ^i , which is characterized by five variables: $(q^i, \zeta^i, s_1^i, s_2^i, s_3^i)$. Before describing these parameters in detail, we present the MPC controller.

At each time step t , for every car $i \in \mathcal{N}$, the MPC controller is determined by solving an optimization problem (cf. (1)) over a planning horizon of K steps, indexed by k . The optimization is parameterized by: (a) the longitudinal and lateral positions on the reference trajectory (to be defined shortly), denoted by $(p_{x|t}^{\text{ref},i,k}, p_{y|t}^{\text{ref},i,k})_{k \in [K]}$; (b) the current state of car i at time t , $\mathbf{x}_t^i$; and (c) the longitudinal and lateral positions of the opponent cars located just behind and just ahead of car i . We use the notation $j^*, j_* \in \mathcal{N}$ to denote the car in front and the car behind car i at the start of the planning window, based on longitudinal position. For any opponent vehicle $j \in \{j^*, j_*\}$, we assume that the lateral velocity is zero during the planning horizon for the MPC controller, and the longitudinal velocity remains constant at its value at the start of the planning horizon. That is, for every $k \in [K]$, $p_{y|t}^{j,k} = p_{y,t}^j$, and $p_{x|t}^{j,k+1} = p_{x|t}^{j,k} + \Delta t \cdot v_{x,t}^j$. With this setup, we can now describe the MPC optimization problem:

$$\min_{(\mathbf{x}_t^{i,k})_{k=1}^K, (\mathbf{u}_t^{i,k})_{k=0}^{K-1}} \sum_{k=1}^K \left\| \begin{pmatrix} p_{x|t}^{i,k} - p_{x|t}^{\text{ref},i,k} \\ p_{y|t}^{i,k} - p_{y|t}^{\text{ref},i,k} \end{pmatrix} \right\|_{\mathbf{Q}}^2 + \sum_{k=1}^{K-1} \left\| \begin{pmatrix} d_t^{i,k} - d_t^{i,k-1} \\ \delta_t^{i,k} - \delta_t^{i,k-1} \end{pmatrix} \right\|_{\mathbf{R}}^2 \quad (1a)$$

$$\text{s.t. } \mathbf{x}_t^{i,k+1} = f^i(\mathbf{x}_t^k, \mathbf{u}_t^{i,k}), \quad \forall k = 0, 1, \dots, K-1, \quad (1b)$$

$$\mathbf{x}_t^{i,0} = \mathbf{x}_t^i, \quad (1c)$$

$$\Delta \delta_{\min}^i \leq \delta_t^{i,k} - \delta_t^{i,k-1} \leq \Delta \delta_{\max}^i, \quad \forall k = 0, 1, \dots, K-1, \quad (1d)$$

$$d_{\min}^i \leq d_t^{i,k} \leq d_{\max}^i, \quad \forall k = 0, 1, \dots, K-1, \quad (1e)$$

$$|p_{y,t}^{i,k}| \leq w_{\max}/2, \quad \forall k = 1, \dots, K, \quad (1f)$$

$$|p_{y|t}^{i,k} - p_{y|t}^{j,k}| \geq p_y^{\min}, \quad \forall k = 1, \dots, K, \forall j \neq i, \quad (1g)$$

$$|p_{x|t}^{i,k} - p_{x|t}^{j,k}| \geq p_x^{\min}, \quad \forall k = 1, \dots, K, \forall j \neq i, \quad (1h)$$

where $\mathbf{x}_t^{i,k} = (p_{x|t}^{i,k}, p_{y|t}^{i,k}, \phi_t^{i,k}, v_{x|t}^{i,k}, v_{y|t}^{i,k}, \omega_t^{i,k})$ is the state of car i in the Frenet frame at the k^{th} step in the planning horizon; $\mathbf{u}_t^{i,k} = (d_t^{i,k}, \delta_t^{i,k})$ is the control input of car i at the k^{th} step in the planning horizon; $w_{\max}$ is the track length, and $p_x^{\min}$ and $p_y^{\min}$ are the minimum required separation between two cars in the longitudinal and lateral directions, respectively; $d_{\min}^i$ and $d_{\max}^i$ are the throttle limits, and $\Delta \delta_{\min}^i$ and $\Delta \delta_{\max}^i$ are the steering rate limits; and $\mathbf{Q}$ and $\mathbf{R}$ are positive definite matrices. Following the MPC approach, the control input at time t is then $\mathbf{u}_t^{i,0}$.

In (1), (1a) defines the MPC objective, where the first term penalizes the tracking error relative to the reference trajectory, and the second term penalizes variations in the control input. (1b) represents the system dynamics constraint, while (1c) ensures the planning horizon begins at the car's current state. (1d) and (1e) enforce constraints on the control inputs, including throttle limits and steering rate bounds. (1f) ensures the car stays within the track boundaries, and (1g) and (1h) enforce the minimum separation between vehicles in the lateral and longitudinal directions, respectively. Next, we describe the policy parameter $\theta^i = (q^i, \zeta^i, s_1^i, s_2^i, s_3^i) \in \mathbb{R}^5$, which parametrizes the policy.

(i) **Parameter q^i :** In (1a), we take $\mathbf{R} = \mathbf{I}$ and $\mathbf{Q} = q^i \cdot \mathbf{I}$, where $\mathbf{I}$ is the identity matrix. A higher q^i value results in a more aggressive controller that closely follows the racing line, but it can introduce oscillations that may increase lap time. In contrast, a lower q^i value allows for smoother merging with reduced oscillations, though it may result in larger lateral errors, increased time loss at corners, and a higher risk of track boundary violations.

(ii) **Parameter ζ^i** : We use this parameter to develop a perturbed version of the optimal (single-agent) race-line (see Supplementary for a discussion on race-line), which is generated by sampling points along the optimal racing line at time intervals of Δt . More formally, let the optimal race-line be denoted by $\mathbf{x}^{\text{rl},i}$. Given the current state $\mathbf{x}_t^i$, we find the closest point on the race-line and consider a race-line starting at time t , denoted by $\mathbf{x}_t^{\text{rl},i}$. Using this, we compute a trajectory of length K , denoted by $(p_{x|t}^{\text{pert},i,k}, p_{y|t}^{\text{pert},i,k})_{k \in [K]}$. Specifically, for every $k \in [K]$, we construct:

$$p_{x|t}^{\text{pert},i,k} = p_{x|t}^{\text{pert},i,k-1} + v_{x|t}^{\text{pert},i,k-1} \cdot \Delta t, \quad p_{y|t}^{\text{pert},i,k} = p_{y|t}^{\text{pert},i,k-1} + v_{y|t}^{\text{pert},i,k-1} \cdot \Delta t,$$

where $v_{x|t}^{\text{pert},i,k-1} = \zeta^i \bar{v}_{x,t+k-1}^{\text{rl},i}$ and $v_{y|t}^{\text{pert},i,k-1} = \zeta^i \bar{v}_{y,t+k-1}^{\text{rl},i}$ are the perturbed race-line velocities. Higher values of ζ^i capture how aggressively the vehicle wants to follow the optimal race-line.

(iii) **Parameters s_1, s_2, s_3** : The reference trajectory, $(p_{x|t}^{\text{ref},i,k}, p_{y|t}^{\text{ref},i,k})$, is generated by modifying the perturbed race-line $(p_{x|t}^{\text{pert},i,k}, p_{y|t}^{\text{pert},i,k})_{k \in [K]}$ by accounting for the positions and velocities of the cars immediately ahead (in terms of longitudinal coordinates) of the ego car and immediately behind (in terms of longitudinal coordinates) ego car. Let's denote the ego car by i , the car immediately ahead of this car by j^* , and the one immediately behind by j_* . We define the reference trajectory as follows $p_{y|t}^{\text{ref},i,k} = \text{clip}(p_{y|t}^{\text{pert},i,k} + p_{y|t}^{\text{ot},i,k} + p_{y|t}^{\text{bl},i,k}, [-w_{\max}, w_{\max}])$, where

$$p_{y|t}^{\text{ot},i,k} = \text{sign}(p_{y,t}^i - p_{y,t}^{j^*}) \max \left\{ (s_1 - |(p_{y,t}^i - p_{y,t}^{j^*})|) \exp \left(-s_2 \left(\Delta p_{x|t}^{i,j^*,k} \right)^2 \right), 0 \right\} \\ + \text{sign}(p_{y,t}^i - p_{y,t}^{j_*}) \max \left\{ (s_1 - |(p_{y,t}^i - p_{y,t}^{j_*})|) \exp \left(-s_2 \left(\Delta p_{x|t}^{i,j_*,k} \right)^2 \right), 0 \right\},$$

is an adjustment for overtaking that smoothly changes the trajectory of the ego vehicle opposite the leading vehicle to overtake. Here, for any $j \in \{j^*, j_*\}$, $\Delta p_{x|t}^{i,j,k} = p_{x|t}^{\text{pert},i,k} - p_{x|t}^{j,k}$, and

$$\bar{p}_{y|t}^{\text{bl},i,k} = \mathbb{I}(v_{x|t}^{\text{pert},i,k} \leq v_{x,t}^{j_*}) \mathbb{I}(p_{x|t}^{\text{pert},i,k} \geq p_{x|t}^{j_*,k}) h(p_{y|t}^{j_*,k}, p_{x|t}^{\text{pert},i,k}, v_{x|t}^{\text{pert},i,k}, v_{x,t}^{j_*}, \Delta p_{x|t}^{i,j_*,k}) \\ + \mathbb{I}(v_{x|t}^{\text{pert},i,k} \leq v_{x,t}^{j^*}) \mathbb{I}(p_{x|t}^{\text{pert},i,k} \geq p_{x|t}^{j^*,k}) h(p_{y|t}^{j^*,k}, p_{x|t}^{\text{pert},i,k}, v_{x|t}^{\text{pert},i,k}, v_{x,t}^{j^*}, \Delta p_{x|t}^{i,j^*,k})$$

is the adjustment for blocking that smoothly changes the trajectory of the ego vehicle towards trailing vehicle to block it. Here, for any $j \in \{j^*, j_*\}$,

$$h(p_{y|t}^{j,k}, p_{x|t}^{\text{pert},i,k}, v_{x|t}^{\text{pert},i,k}, v_{x,t}^j, \Delta p_{x|t}^{i,j,k}) \\ = (p_{y|t}^{j,k} - p_{y|t}^{\text{pert},i,k}) (1 - \exp(-s_3(v_{x|t}^{\text{pert},i,k} - v_{x,t}^j))) \exp(-s_2(\Delta p_{x|t}^{i,j,k})^2).$$

One-step Utility Function. We consider that the one-step utility for every car is to maximize progress along track: $\mathbb{R} \ni r^i(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1}) = (p_{x,t+1}^i - \max_{j \neq i} p_{x,t+1}^j) - (p_{x,t}^i - \max_{j \neq i} p_{x,t}^j)$.

3. Approximating Multi-agent Interaction

In this section, we provide a tractable approach to compute an approximate Nash equilibrium for the racing game described in Section 2. Core to our approach is the framework of α -potential functions, recently introduced in (Guo et al., 2023; Maheshwari et al., 2024; Guo et al., 2024).

3.1. α -Potential Function

Definition 2 A potential function $\Phi : \mathcal{X} \times \Theta \rightarrow \mathbb{R}$ is called a dynamic α -potential function² with approximation parameter α if for every $\mathbf{x} \in \mathcal{X}, i \in \mathcal{N}, \theta \in \Theta, \theta^{i'} \in \Theta^i$,

$$|(\Phi(\mathbf{x}, \theta^i, \theta^{-i}) - \Phi(\mathbf{x}, \theta^{i'}, \theta^{-i})) - (V^i(\mathbf{x}, \theta^i, \theta^{-i}) - V^i(\mathbf{x}, \theta^{i'}, \theta^{-i}))| \leq \kappa. \quad (2)$$

This definition intuitively requires that for any agent, the change in its value function resulting from a unilateral adjustment to its policy parameter can be closely approximated by the corresponding change in the dynamic α -potential function. This property allows us to approximate the Nash equilibrium as an optimizer of the near-potential function (as proved in the supplementary material).

Proposition 3 Given an α -potential function Φ , for any $\mathbf{x} \in \mathcal{X}, \lambda > 0$, and any policy θ^* satisfying $\Phi(\mathbf{x}, \theta^*) \geq \max_{\theta \in \Theta} \Phi(\mathbf{x}, \theta) - \lambda$, the policy θ^* constitutes a $(\lambda + \alpha)$ -approximate Nash equilibrium.

3.2. Computational Approach

Our approach for real-time approximation of Nash equilibrium relies on two phases: offline and online. In the offline phase, we learn an α -potential function using simulated game data. In the online phase, the ego vehicle updates its policy parameters by optimizing the potential function.

Offline Phase: We parameterize the potential function through using a feed-forward neural network with ReLU activation and a BatchNorm layer added at the beginning. More concretely, we define the parametrized potential function as $\Phi(\cdot; \phi) : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$, where ϕ denotes the weights of neural network. Using Definition 2, we cast the problem of learning potential function as a semi-infinite program as shown below:

$$\begin{aligned} \min_{y, \phi} \quad & y \\ \text{s.t.} \quad & \left| (\Phi(\mathbf{x}, \theta^i, \theta^{-i}; \phi) - \Phi(\mathbf{x}, \theta^{i'}, \theta^{-i}; \phi)) - (V^i(\mathbf{x}, \theta^i, \theta^{-i}; v) - V^i(\mathbf{x}, \theta^{i'}, \theta^{-i}; v)) \right| \leq y, \\ & \forall i \in I, \forall \theta^i, \theta^{i'} \in \Theta^i, \forall \theta^{-i} \in \Theta^{-i}, \mathbf{x} \in \mathcal{X}, \end{aligned} \quad (3)$$

where we use a neural network (same architecture as potential function), with parameters v , to estimate the value function V^i for every $i \in \mathcal{N}$. Let ϕ^* be a solution of the above optimization problem. The main challenge with solving (3) is that there are un-countably many constraints, one constraint corresponding to each value of initial state and policy parameter. Therefore, we numerically solve (3) by using simulated game data with randomly chosen starting position and policy parameters. Details of simulated game are discussed in next section.

Online Phase: Leveraging Proposition 3, the ego vehicle optimizes the learned potential function, i.e. $\Phi(\cdot, \phi^*)$, to approximate the Nash equilibrium policy parameter. More formally, given the current state $\mathbf{x}_t$, the ego vehicle computes $\theta^* \in \arg \max_{\theta \in \Theta} \Phi(\mathbf{x}_t, \theta; \phi^*)$, using a non-linear optimization solver. Using θ^* , the ego vehicle takes action $\mathbf{u}_t^i = \pi^i(\mathbf{x}_t; \theta^{*,i})$.

4. Numerical Evaluation

Here, we evaluate our approach on a numerical simulator by focusing on three questions: **(Q1)** Can our approach closely approximate Nash equilibrium and generate competitive behavior? **(Q2)** How

2. For convenience, we adopt a slightly different definition of the α -potential function than that in Guo et al. (2023); Maheshwari et al. (2024); however, the results of this work extend to the definitions used there.

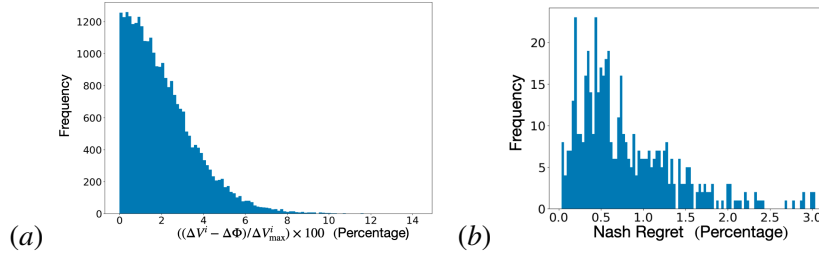


Figure 1: Histogram of (a) Relative approximation gap of potential function (b) Nash regret

do hyper-parameters like the discount factor γ and the amount of data used to learn the α -potential function affect the performance? **(Q3)** How does our approach compare against common baselines?

Experimental Setup: We generate a dataset of 4000 races, each lasting 50 seconds, conducted with randomly chosen policy parameters and involving 3 cars. This dataset is used to first train value function estimators V^1 , V^2 , and V^3 for each of the cars. These are then used to learn Φ using (3). To maximize the learned potential function, we use gradient ascent with a learning rate of 10^{-4} and warm-start by using the solution from the previous time step.

4.1. Competitive Behavior by Approximating Nash Equilibrium

We observe that the approximation gap of the learned potential function is small. As shown in Figure 1(a), the approximation gap across all states and policy parameters used in the training samples remains within 10% of the value function’s range, with a median gap of approximately 2%. Next, we demonstrate that the optimization solver effectively computes the maximizer of the potential function, leading to a lower Nash approximation error. In particular, Figure 1(b) shows the *Nash regret* for the ego agent, defined as $\max_{\theta^i} V^i(\mathbf{x}, \theta^i, \theta^{*, -i}) - V^i(\mathbf{x}, \theta^{*, i}, \theta^{*, -i})$, where θ^* is the optimizer of the potential function with the starting state $\mathbf{x}$. The regret is plotted for different game states during a race, and we observe that it remains within 3% of the range of the value function. In summary, Figure 1 highlights that the dynamic game admits an α -potential function with small α , and that we accurately compute a near-optimal solution to the potential function.

Next, in Figure 2, we show that, by fixing all policy parameters but one, the parameter that (approx.) maximizes potential function generates a trajectory that maximizes progress along the track over the next 5 seconds. This is highlighted by yellow diamond in Figure 2.

4.2. Performance Comparison

To address questions **Q2** and **Q3**, we conduct 99 races involving three agents: **ego**, O_1 , and O_2 . Here, **ego** represents the agent using our proposed algorithm, while O_1 and O_2 are opponents employing other algorithms. In this study, we vary the opponents’ algorithms and compute the winning fraction for the **ego** car. A visualization of the track used in the study, along with the computed optimal race-line, is provided in the Appendix (Figure 4). The starting positions for the races are taken from three regions (denoted by R_1 , R_2 , and R_3), such that R_1 is the furthest ahead on the track, followed by R_2 , and then R_3 , as shown in the Appendix (Figure 4). We perform 33 races, each with the **ego** agent starting in R_1 , R_2 , or R_3 . Also, O_1 is always placed ahead of O_2 .

For the **ego**, we use a discount factor $\gamma = 0.99$ and a training dataset comprising 4000 races. Below, we summarize five different opponent strategies for O_1 and O_2 :

- (i) **Case I:** Opponents trained with a lower discount factor than **ego** (i.e., $\gamma = 0.98$);
- (ii) **Case II:** Opponents trained with a higher discount factor than **ego** (i.e., $\gamma = 0.995$);
- (iii) **Case III:** Opponents trained using fewer simulated races than **ego** (i.e., 400 races);

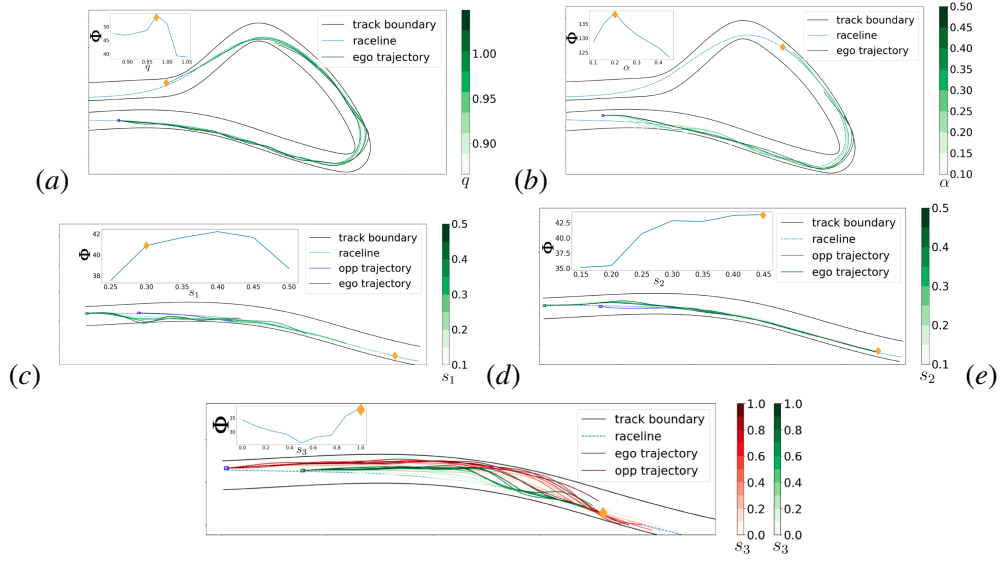


Figure 2: Potential values and the trajectories at a given joint state for different (a) q (b) ζ (c) s_1 (d) s_2 (e) s_3 of only the ego agent. We only denote 2 players here (only 1 player for (a) and (b)) and the 3rd player is far away from this position to not affect any players. Additionally, for ease of readability, we only show the impact of variation in trajectory of other player in response to ego in (e) as such deviations are not significant in (c) and (d).

(iv) **Case IV:** Opponents use the *Iterated Best Response* (IBR) algorithm³, which computes the best response of opponents in round-robin for a fixed number of rounds (i.e., 6), with a planning horizon⁴ of length $2s$;

(v) **Case V:** Opponents trained using self-play RL⁵. We use a similar observation and reward as used in Thakkar et al. (2022) and train with $2M$ steps.

The number of races won for all cases are provided in Table 1, where we see that ego agent trained using our approach has superior performance in comparison to all other opponent strategies.

Performance Variation with Hyperparameters (Cases I–III). The ego car outperforms opponents with lower discount factors (**Case I**) because lower discount factors lead to more myopic behavior, causing opponents to prioritize short-term progress over long-term track performance. Similarly, the ego car also outperforms opponents with higher discount factors (**Case II**) as higher discount factors increase the “effective horizon” of the game, which requires significantly more data to accurately approximate the value and potential functions. Furthermore, the ego car has superior performance against opponents trained with less data of only 400 races (**Case III**), as more data enables the learning of a more accurate α -potential function.

Comparison with opponents using IBR (Case IV). Our approach surpasses IBR for two key reasons. First, IBR may not always converge to a Nash equilibrium. Second, IBR’s real-time computation is often restricted to local planning with a short $2s$ horizon, as noted in Spica et al. (2020); Wang et al. (2019a). In contrast, competitive racing demands prioritizing a global racing line to optimize long-term performance, with strategic deviations for overtaking or blocking rather

3. The IBR algorithm used here is representative of the methods developed in Spica et al. (2020); Wang et al. (2020b), though it may not be an exact implementation, as the original code is not publicly available.

4. Choice of hyperparameters is such that it roughly takes the same compute time as our approach.

5. Here, self-play RL represents the approach in Thakkar et al. (2022), excluding the high-level tactical planner.

Racing Scenario	# wins (ours)	# wins (O_1)	# wins (O_2)
Case I (Opponents with low γ)	61	28	10
Case II (Opponents with high γ)	52	40	7
Case III (Opponents trained with less data)	76	16	7
Case IV (Opponents using IBR)	73	22	4
Case V (Opponents trained using self-play RL)	91	7	1

Table 1: Outcomes of 99 races conducted between 3 cars under three different initial positions

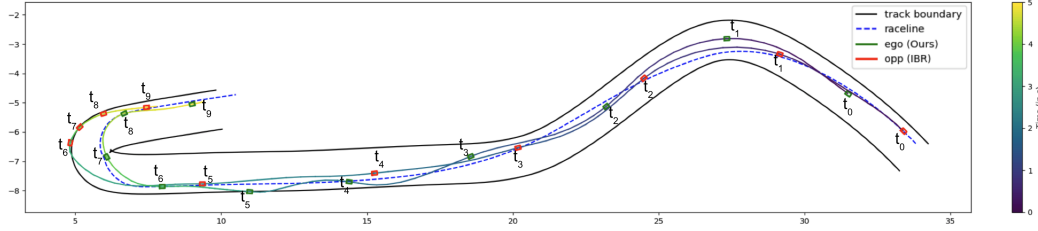


Figure 3: Example overtake in a race MPC vs IBR. Opponent (IBR) overtakes from t_4 to t_5 but later suffers at the turn from t_5 to t_9 when the Ego agent (Ours) overtakes back to re-claim it’s position

than short-term gains. This distinction is highlighted in Figure 3, where the IBR player achieves higher straight-line speeds to overtake but struggles in turns due to aggressive braking. While our implementation is not a direct comparison with Spica et al. (2020), our approach supports longer-horizon planning for real-time control by leveraging offline potential function learning. This is achieved by constraining the policy space to primitive behaviors, closely aligning with practical racing strategies.

Comparison with opponents using self-play RL (Case V). Our approach also outperforms opponents trained via self-play RL. While we do not claim an optimized implementation of self-play RL—acknowledging potential improvements through further training, hyperparameter tuning, or tactical enhancements as in Thakkar et al. (2022)—our method offers clear advantages. It delivers interpretable solutions grounded in realistic racing strategies and approximation guarantees to a Nash equilibrium in races with more than 2 cars. In contrast, self-play RL often requires additional insights to develop effective policies. For example, Thakkar et al. (2022) showed that augmenting self-play RL with high-level tactical plans significantly enhances policy learning.

5. Conclusion

In this work, we study real-time algorithms for competitive multi-car autonomous racing. Our approach is built on two key contributions: first, a novel policy parametrization and utility function that effectively capture competitive racing behavior; second, the use of dynamic α -potential functions to develop real-time algorithms that approximate Nash equilibrium strategies. This framework enables the learning of equilibrium strategies over long horizons at different game states through the maximization of a potential function.

There are several directions of future research. One interesting direction is to integrate the strengths of self-play RL discussed in Thakkar et al. (2022), with our approach. For instance, potential function maximization could guide self-play RL as a high-level reference plan generator. Alternatively, our method could initialize self-play policies via behavior cloning or refined reward design, such as using the potential function’s maximum value as a reward signal or leveraging GAIL Ho and Ermon (2016) with expert trajectories derived from our approach.

References

- Bharathan Balaji, Sunil Mallya, Sahika Genc, Saurabh Gupta, Leo Dirac, Vineet Khare, Gourav Roy, Tao Sun, Yunzhe Tao, Brian Townsend, et al. Deepracer: Autonomous racing platform for experimentation with sim2real reinforcement learning. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 2746–2754. IEEE, 2020.
- Johannes Betz, Hongrui Zheng, Alexander Liniger, Ugo Rosolia, Phillip Karle, Madhur Behl, Venkat Krovi, and Rahul Mangharam. Autonomous vehicles on the edge: A survey on autonomous vehicle racing. *IEEE Open Journal of Intelligent Transportation Systems*, 3:458–488, 2022.
- Lukas Brunke. Learning model predictive control for competitive autonomous racing. *ArXiv*, abs/2005.00826, 2020. URL <https://api.semanticscholar.org/CorpusID:218487458>.
- Alexander Buyval, Aidar Gabdulin, Ruslan Mustafin, and Ilya Shimchik. Deriving overtaking strategy from nonlinear model predictive control for a race car. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 2623–2628. IEEE, 2017.
- Simon Le Cleac’h, Mac Schwager, and Zachary Manchester. Algames: A fast solver for constrained dynamic games. *arXiv preprint arXiv:1910.09713*, 2019.
- David Fridovich-Keil, Ellis Ratner, Lasse Peters, Anca D Dragan, and Claire J Tomlin. Efficient iterative linear-quadratic approximations for nonlinear multi-player general-sum differential games. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 1475–1481. IEEE, 2020.
- Xin Guo, Xinyu Li, Chinmay Maheshwari, Shankar Sastry, and Manxi Wu. Markov α -potential games: Equilibrium approximation and regret analysis. *arXiv preprint arXiv:2305.12553*, 3, 2023.
- Xin Guo, Xinyu Li, and Yufei Zhang. An α -potential game framework for n -player dynamic games. *arXiv preprint arXiv:2403.16962*, 2024.
- Alexander Heilmeyer, Alexander Wischnewski, Leonhard Hermansdorfer, Johannes Betz, Markus Lienkamp, and Boris Lohmann. Minimum curvature trajectory planning and control for an autonomous race car. *Vehicle System Dynamics*, 58(10):1497–1527, 2020. doi: 10.1080/00423114.2019.1631455.
- James Herman, Jonathan Francis, Siddha Ganju, Bingqing Chen, Anirudh Koul, Abhinav Gupta, Alexey Skabelkin, Ivan Zhukov, Max Kumskey, and Eric Nyberg. Learn-to-race: A multimodal control environment for autonomous racing. In *proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9793–9802, 2021.
- Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Neural Information Processing Systems*, 2016. URL <https://api.semanticscholar.org/CorpusID:16153365>.

- Achin Jain and Manfred Morari. Computing the racing line using Bayesian optimization. *2020 59th IEEE Conference on Decision and Control (CDC)*, pages 6192–6197, 2020.
- Yixuan Jia, Maulik Bhatt, and Negar Mehr. Rapid: Autonomous multi-agent racing using constrained potential dynamic games. In *2023 European Control Conference (ECC)*, pages 1–8. IEEE, 2023.
- Chanyoung Jung, Seungwook Lee, Hyunki Seong, Andrea Finazzi, and David Hyunchul Shim. Game-theoretic model predictive control with data-driven identification of vehicle model for head-to-head autonomous racing. *arXiv preprint arXiv:2106.04094*, 2021.
- Juraj Kabzan, Miguel I Valls, Victor JF Reijgwart, Hubertus FC Hendriks, Claas Ehmke, Manish Prajapat, Andreas Bühler, Nikhil Gosala, Mehak Gupta, Ramya Sivanesan, et al. Amz driverless: The full autonomous racing system. *Journal of Field Robotics*, 37(7):1267–1294, 2020.
- Dvij Kalaria, Qin Lin, and John M. Dolan. Adaptive planning and control with time-varying tire models for autonomous racing using extreme learning machine. *ArXiv*, abs/2303.08235, 2023a. URL <https://api.semanticscholar.org/CorpusID:257532643>.
- Dvij Kalaria, Qin Lin, and John M Dolan. Towards optimal head-to-head autonomous racing with curriculum reinforcement learning. *arXiv preprint arXiv:2308.13491*, 2023b.
- Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 620(7976):982–987, 2023.
- Talha Kavuncu, Ayberk Yaraneri, and Negar Mehr. Potential ilqr: A potential-minimizing controller for planning multi-agent interactive trajectories. *arXiv preprint arXiv:2107.04926*, 2021.
- Forrest Laine, David Fridovich-Keil, Chih-Yuan Chiu, and Claire Tomlin. Multi-hypothesis interactions in game-theoretic motion planning. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8016–8023. IEEE, 2021.
- Alexander Liniger and John Lygeros. A noncooperative game approach to autonomous racing. *IEEE Transactions on Control Systems Technology*, 28(3):884–897, 2019.
- Alexander Liniger, Alexander Domahidi, and Manfred Morari. Optimization-based autonomous racing of 1: 43 scale rc cars. *Optimal Control Applications and Methods*, 36(5):628–647, 2015.
- Xinjie Liu, Lasse Peters, and Javier Alonso-Mora. Learning to play trajectory games against opponents with unknown objectives. *IEEE Robotics and Automation Letters*, 8(7):4139–4146, 2023.
- Chinmay Maheshwari, Manxi Wu, and Shankar Sastry. Decentralized learning in general-sum Markov games. *IEEE Control Systems Letters*, 2024.
- Gennaro Notomista, Mingyu Wang, Mac Schwager, and Magnus Egerstedt. Enhancing game-theoretic autonomous car racing using control barrier functions. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 5393–5399. IEEE, 2020.

- Lasse Peters, Andrea Bajcsy, Chih-Yuan Chiu, David Fridovich-Keil, Forrest Laine, Laura Ferranti, and Javier Alonso-Mora. Contingency games for multi-agent interaction. *IEEE Robotics and Automation Letters*, 2024.
- Ugo Rosolia and Francesco Borrelli. Learning how to autonomously race a car: A predictive control approach. *IEEE Transactions on Control Systems Technology*, 28:2713–2719, 2020.
- Ugo Rosolia, Ashwin Carvalho, and Francesco Borrelli. Autonomous racing using learning model predictive control. In *2017 American control conference (ACC)*, pages 5115–5120. IEEE, 2017.
- Dorsa Sadigh, Shankar Sastry, Sanjit A Seshia, and Anca D Dragan. Planning for autonomous cars that leverage effects on human actions. In *Robotics: Science and systems*, volume 2, pages 1–9. Ann Arbor, MI, USA, 2016.
- Wilko Schwarting, Alyssa Pierson, Sertac Karaman, and Daniela Rus. Stochastic dynamic games in belief space. *IEEE Transactions on Robotics*, 37(6):2157–2172, 2021.
- Aman Sinha, Matthew O’Kelly, Hongrui Zheng, Rahul Mangharam, John Duchi, and Russ Tedrake. Formulazero: Distributionally robust online adaptation via offline population synthesis. In *International Conference on Machine Learning*, pages 8992–9004. PMLR, 2020.
- Yunlong Song, Mats Steinweg, Elia Kaufmann, and Davide Scaramuzza. Autonomous drone racing with deep reinforcement learning. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1205–1212. IEEE, 2021.
- Riccardo Spica, Eric Cristofalo, Zijian Wang, Eduardo Montijano, and Mac Schwager. A real-time game theoretic planner for autonomous two-player drone racing. *IEEE Transactions on Robotics*, 36(5):1389–1403, 2020.
- Rishabh Saumil Thakkar, Aryaman Singh Samyals, David Fridovich-Keil, Zhe Xu, and Ufuk Topcu. Hierarchical control for head-to-head autonomous racing. *arXiv preprint arXiv:2202.12861*, 2022.
- Rishabh Saumil Thakkar, Aryaman Singh Samyals, David Fridovich-Keil, Zhe Xu, and Ufuk Topcu. Hierarchical control for cooperative teams in competitive autonomous racing. *IEEE Transactions on Intelligent Vehicles*, 2024.
- Richard W. Vesel. Racing line optimization @ race optimal. *ACM SIGEVolution*, 7:12 – 20, 2015.
- Mingyu Wang, Zijian Wang, John Talbot, J Christian Gerdes, and Mac Schwager. Game theoretic planning for self-driving cars in competitive scenarios. In *Robotics: Science and Systems*, pages 1–9, 2019a.
- Mingyu Wang, Negar Mehr, Adrien Gaidon, and Mac Schwager. Game-theoretic planning for risk-aware interactive agents. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6998–7005. IEEE, 2020a.
- Mingyu Wang, Zijian Wang, John Talbot, J Christian Gerdes, and Mac Schwager. Game-theoretic planning for self-driving cars in multivehicle competitive scenarios. *IEEE Transactions on Robotics*, 37(4):1313–1325, 2021.

- Zijian Wang, Riccardo Spica, and Mac Schwager. Game theoretic motion planning for multi-robot racing. In *Distributed Autonomous Robotic Systems: The 14th International Symposium*, pages 225–238. Springer, 2019b.
- Zijian Wang, Tim Taubner, and Mac Schwager. Multi-agent sensitivity enhanced iterative best response: A real-time game theoretic planner for drone racing in 3d environments. *Robotics Auton. Syst.*, 125:103410, 2020b. URL <https://api.semanticscholar.org/CorpusID:211050763>.
- Grady Williams, Brian Goldfain, Paul Drews, James M Rehg, and Evangelos A Theodorou. Autonomous racing with autorally vehicles and differential games. *arXiv preprint arXiv:1707.04540*, 2017.
- Peter R Wurman, Samuel Barrett, Kenta Kawamoto, James MacGlashan, Kaushik Subramanian, Thomas J Walsh, Roberto Capobianco, Alisa Devlic, Franziska Eckert, Florian Fuchs, et al. Out-racing champion gran turismo drivers with deep reinforcement learning. *Nature*, 602(7896): 223–228, 2022.
- Hongrui Zheng, Zhijun Zhuang, Johannes Betz, and Rahul Mangharam. Game-theoretic objective space planning. *arXiv preprint arXiv:2209.07758*, 2022.
- Edward L Zhu and Francesco Borrelli. A sequential quadratic programming approach to the solution of open-loop generalized nash equilibria. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3211–3217. IEEE, 2023.
- Edward L Zhu and Francesco Borrelli. A sequential quadratic programming approach to the solution of open-loop generalized nash equilibria for autonomous racing. *arXiv preprint arXiv:2404.00186*, 2024.

Appendix A. Proof of Proposition 3

Consider a policy parameter θ^* such that $\Phi(\mathbf{x}, \theta^*) \geq \max_{\theta \in \Theta} \Phi(\mathbf{x}, \theta) - \lambda$. For any $\theta^i \in \Theta^i$,

$$\begin{aligned} V^i(\mathbf{x}, \theta^{*,i}, \theta^{*, -i}) - V^i(\mathbf{x}, \theta^i, \theta^{*, -i}) &\geq \Phi(\mathbf{x}, \theta^{*,i}, \theta^{*, -i}) - \Phi(\mathbf{x}, \theta^i, \theta^{*, -i}) - \alpha \\ &\geq -\lambda - \alpha, \end{aligned}$$

where first inequality is due to (2) and the second inequality is because θ^* maximizes Φ . The proof follows using the definition of Nash equilibrium (Definition 1).

Appendix B. Description of Single-agent Racing Line

Race drivers follow a racing line for specific maneuvers. This line can be used as a reference path by the motion planner to assign time-optimal trajectories while avoiding collision. The racing line is minimum-time or minimum-curvature. They are similar, but the minimum-curvature path additionally allows the highest cornering speeds given the maximum legitimate lateral acceleration [Heilmeier et al. \(2020\)](#).

There are many proposed solutions to finding the optimal racing line, including nonlinear optimization [Rosolia and Borrelli \(2020\)](#); [Heilmeier et al. \(2020\)](#), genetic algorithm-based search [Vesel \(2015\)](#) and Bayesian optimization [Jain and Morari \(2020\)](#). However, for our work, we calculate the minimum-curvature optimal line, which is close to the optimal racing line as proposed by [Heilmeier et al. \(2020\)](#). The race track is represented by a sequence of tuples (x_i, y_i, w_i) , $i \in \{0, \dots, N-1\}$, where (x_i, y_i) denotes the coordinate of the center location and w_i denotes the lane width at the i -th point. The output racing line consists of a tuple of seven variables: coordinates x and y , longitudinal displacement s , longitudinal velocity v_x , acceleration a_x , heading angle ψ , and curvature κ . It is obtained by minimizing the following cost:

$$\begin{aligned} \min_{\eta_1 \dots \eta_N} \quad & \sum_{n=0}^{N-1} \kappa_i^2(n) \\ \text{s.t.} \quad & \eta_i \in \left[-\frac{w_i}{2} + \frac{w_{veh}}{2}, \frac{w_i}{2} - \frac{w_{veh}}{2} \right] \end{aligned} \tag{4}$$

where the vehicle width is w_{veh} , and η_i is the lateral displacement with respect to the reference center line.

To create a velocity profile, we need to consider the vehicle's constraints on both longitudinal and lateral acceleration [Heilmeier et al. \(2020\)](#). Our approach involves generating a library of velocity profiles, each tailored to specific lateral acceleration limits determined by the friction coefficients for the front (μ_f) and rear (μ_r) tires, as well as the vehicle's mass (m) and the gravitational constant (g). In particular, we produce a set of velocity profiles covering a range of maximum lateral forces corresponding to the friction μ_{eff} within the interval $[\mu_{min}, \mu_{max}]$. This library allows us to retrieve a velocity profile that matches a given value of μ . Interpolation is necessary when we encounter a friction value that falls within the valid range but is not explicitly present in the library.

An example of a racing line calculated for the racetrack used in our numerical study in Section 4 is shown in Figure 4.

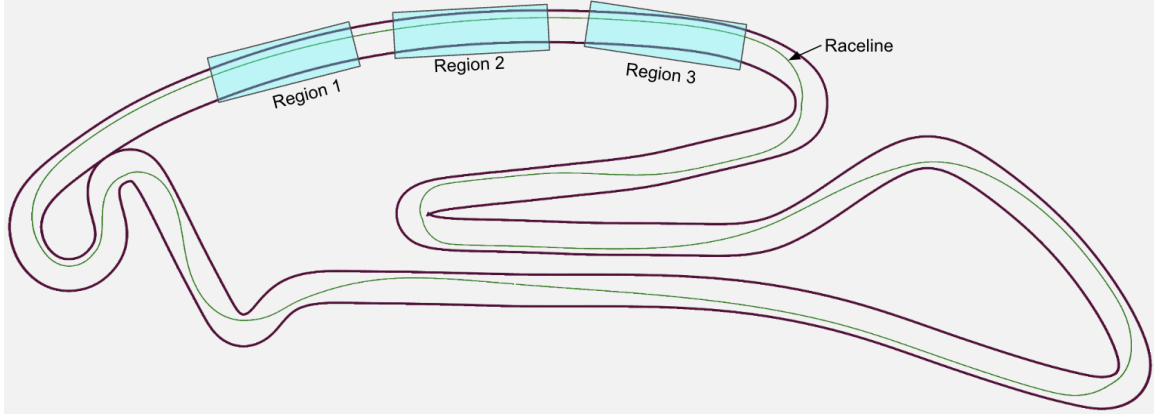


Figure 4: Track and the starting position regions

Appendix C. Dynamic Bicycle Model

For any car i , we denote its mass by m^i , its moment of inertia in the vertical direction about the center of mass by I_z^i , the distance between the center of mass (COM) and its front wheel by l_f^i , and the distance from the COM to the rear wheel l_r^i . Also, κ_t^i denotes the inverse of radius of curvature of the track at $p_{x,t}^i$. Using these notations, the dynamics of car i is defined below:

$$\begin{bmatrix} p_{x,t+1}^i \\ p_{y,t+1}^i \\ \phi_{t+1}^i \\ \tilde{v}_{x,t+1}^i \\ \tilde{v}_{y,t+1}^i \\ \omega_{t+1}^i \end{bmatrix} = \begin{bmatrix} p_{x,t}^i \\ p_{y,t}^i \\ \phi_t^i \\ v_{x,t}^i \\ v_{y,t}^i \\ \omega_t^i \end{bmatrix} + \Delta t \begin{bmatrix} v_{x,t}^i \\ v_{y,t}^i \\ \omega_t^i - \frac{\kappa_t^i}{(1-\kappa_t^i p_{y,t}^i)} (\tilde{v}_{x,t}^i \cos(\phi_t^i) - \tilde{v}_{y,t}^i \sin(\phi_t^i)) \\ \frac{1}{m^i} (F_{r,x,t}^i - F_{f,y,t}^i \sin(\delta_t^i) + m^i \tilde{v}_{y,t}^i \omega_t^i) \\ \frac{1}{m^i} (F_{r,y,t}^i + F_{f,y,t}^i \cos(\delta_t^i) - m^i \tilde{v}_{x,t}^i \omega_t^i) \\ \frac{1}{I_z^i} (F_{f,y,t}^i l_f^i \cos(\delta_t^i) - F_{r,y,t}^i l_r^i) \end{bmatrix}, \quad (5)$$

where (i) $v_{x,t}^i = \frac{1}{(1-\kappa_t^i p_{y,t}^i)} (\tilde{v}_{x,t}^i \cos(\phi_t^i) - \tilde{v}_{y,t}^i \sin(\phi_t^i))$, $v_{y,t}^i = \tilde{v}_{x,t}^i \sin(\phi_t^i) + \tilde{v}_{y,t}^i \cos(\phi_t^i)$ are the velocities in frenet frame; (ii) $\tilde{v}_{x,t}^i, \tilde{v}_{y,t}^i$ are velocities in body frame; (iii) $F_{r,x,t}^i = (C_1 - C_2 \tilde{v}_{x,t}^i) d_t^i - C_3 - C_4 (\tilde{v}_{x,t}^i)^2$ is the longitudinal force on the rear tire at time t . Here, C_1 and C_2 are parameters that govern the longitudinal force generated on the car in response to the throttle command, while C_3 and C_4 are parameters that account for the friction and drag forces acting on the car; (iv) $F_{f,y,t}^i = D_f \sin(C_f \tan^{-1}(B_f \alpha_{f,t}^i))$ is the lateral force on the front tire depending on the slipping angle $\alpha_{f,t}^i$, which is given by $\alpha_{f,t}^i = \delta_t^i - \tan^{-1} \left(\frac{\omega_t^i l_f^i + \tilde{v}_{y,t}^i}{\tilde{v}_{x,t}^i} \right)$. Here B_f, C_f, D_f are the parameters of Pacejka tire model; and (v) $F_{r,y,t}^i = D_r \sin(C_r \tan^{-1}(B_r \alpha_{r,t}^i))$ is the lateral force on the rear tire depending on the slipping angle $\alpha_{r,t}^i$, which is given by $\alpha_{r,t}^i = \tan^{-1} \left(\frac{\omega_t^i l_r^i - \tilde{v}_{y,t}^i}{\tilde{v}_{x,t}^i} \right)$. Here B_r, C_r, D_r are the parameters of Pacejka tire model.

Appendix D. Hyperparameters

D.1. Network architecture

We use a simple feed-forward deep neural network with ReLU activation except for the last layer to represent the value function and the potential function. The network for value function consists of 3 hidden layers with (128, 128, 64) hidden features on each layer. The network for potential function consists of 3 hidden layers with (384, 384, 192) hidden features on each layer.

D.2. Training

We use a learning rate of 0.0001 and train for 50000 epochs (both value functions and potential function). Each race consists of 500 time steps with $dt = 0.1s$, hence 50s race.

Appendix E. Self-play RL training

We use standard PPO training parameters as available in stable_baselines3 with batch size 1024, number of epochs 5, learning rate 0.0005, $\gamma = 0.99$ and 8 environments in parallel. The observation used is the same as the joint state input used for our work for fair comparison. The reward design used is also the same as the utility used in our work. We train for 100K time-steps for each iteration of self-play RL where we switch agents for training for total of 99 times i.e. 33 cycles of training for 3 agents

Appendix F. Iterated Best Response (IBR) hyperparameters

We use $N = 6$ iterations for iterated best response with the same utility as the one used in our work and with horizon length of 2s with 20 time-steps of length $dt = 0.1s$. The solve time with the following parameters is 0.1s which is comparable to the compute time required by our algorithm

Appendix G. Table of Notations

Notation	Description
$\mathbf{x}^i$	State vector of car i , including position, velocity, and angular velocity.
p_x^i, p_y^i	Longitudinal and lateral positions of car i in the global frame.
ϕ^i	Orientation of car i in the global frame.
v_x^i, v_y^i	Longitudinal and lateral velocities of car i in the body frame.
ω^i	Angular velocity of car i in the global frame.
$\mathbf{u}^i = (d^i, \delta^i)$	Control input for car i , where d^i is the throttle and δ^i is the steering angle.
d^i	Throttle input of car i .
δ^i	Steering angle of car i .
$d_{\min}, d_{\max}$	Minimum and maximum throttle limits for car i .
$\delta_{\min}, \delta_{\max}$	Minimum and maximum steering angle limits for car i .
m^i	Mass of car i .
I_z^i	Moment of inertia of car i in the vertical direction about the center of mass.
l_f^i	Distance from the center of mass to the front wheel of car i .
l_r^i	Distance from the center of mass to the rear wheel of car i .

Notation	Description
$F_{r,x,t}^i$	Force applied to the rear wheel of car i in the longitudinal direction at time t .
$F_{r,y,t}^i$	Force applied to the rear wheel of car i in the lateral direction at time t .
$F_{f,y,t}^i$	Force applied to the front wheel of car i in the lateral direction at time t .
Δt	Time step for the simulation.
γ	Discount factor in the utility function.
$\mathbf{x}_t$	State of the system at time t .
$\mathbf{u}_t^i$	Control input of car i at time t .
ω_t^i	Angular velocity of car i at time t .
θ^i	Parameter of car i 's policy.
Θ^i	Set of possible parameters for car i 's policy.
Π^i	Set of possible strategies for car i .
Π	Set of joint strategies for all players (cars).
ϵ	The tolerance used in the definition of ϵ -Nash equilibrium.
$V^i(\mathbf{x}, \theta^i, \theta^{-i})$	The expected long-run utility of car i given the state $\mathbf{x}$ and strategy profile θ .
θ^*	The ϵ -Nash equilibrium strategy profile.